

Table of Contents

[GPIBDefTopic](#)

[gpib_apps.4.01](#)

[gpib_apps.4.02](#)

[gpib_apps.4.03](#)

[gpib_apps.4.04](#)

[gpib_apps.4.05](#)

[gpib_apps.4.06](#)

[gpib_apps.4.07](#)

[gpib_apps.4.08](#)

[gpib_apps.4.09](#)

[gpib_apps.4.10](#)

[gpib_apps.4.11](#)

[gpib_apps.4.12](#)

[gpib_prog.3.1](#)

[gpib_prog.3.2](#)

[gpib_prog.3.3](#)

[gpib_prog.3.4](#)

[gpib_prog.3.5](#)

[gpib_prog.3.6](#)

[gpib_prog.3.7](#)

[gpib_prog.3.8](#)

[gpib_too.2.1](#)

[gpib_too.2.2](#)

[gpib_too.2.3](#)

[gpib_too.2.4](#)

[gpib_too.2.5](#)

GPIBDefTopic

<

>

About the GPIB

About the GPIB

The General Purpose Interface Bus (GPIB) is a digital eight-bit parallel communication interface bus for communication between GPIB controllers and instruments connected to the bus. This section describes IG-XL programming for the General Purpose Interface Bus. GPIB information includes the following sections:

- GPIB Theory of Operation
- GPIB Programming
- GPIB Examples

Related Topics

- GPIB (VBT syntax)
- GPIB Interface (handler interface)

<

>

gpib_apps.4.01

<	>
----------------------	----------------------

GPIB Examples

GPIB Examples

This section contains the following information:

- | | |
|---|--|
| • | Cabling and Configuration Issues |
| • | GPIB Functions |
| • | Using GPIB Functions |

For more information, see:

- | | |
|---|---|
| • | GPIB (VBT Syntax Reference) |
| • | GPIB Programming |

<	>
----------------------	----------------------

gpib_apps.4.02

<	>
-----------------	-----------------

GPIB Examples : Cabling and Configuration Issues

Cabling and Configuration Issues

The GPIB requires strict adherence to the following cable length and configuration specifications:

- | | |
|--|---|
| | <ul style="list-style-type: none">The total number of instruments on the GPIB cannot exceed 15. |
| | <ul style="list-style-type: none">The cable distance between instruments (in meters) cannot be more than twice the number of instruments and cannot exceed 20 meters overall. |

Based on experience, connecting instruments in a daisy-chain arrangement seems to work best, with the incoming signal line being stacked closest to the instrument.

Also, avoid cycling power of instruments on the bus while the bus is in operation. This may cause faulty transfers.

Finally, you can have one-third of the instruments powered off without affecting GPIB operation. More than that can cause faulty transfers or an error message such as, "GPIB timeout".

<	>
-----------------	-----------------

gpib_apps.4.03



[GPIB Examples](#) : GPIB Functions

GPIB Functions

Functions in this section are user defined functions that wrap GPIB Service and GPIB Controller functions. These functions show you how to create functions to wrap GPIB functions.

The following topics are included:

- [Define Constants for Carriage Return and Line Feed](#)
 - [Create Variables to Hold the Read and Write Terminators](#)
 - [ConnectDevice](#)
 - [DisconnectDevice](#)
 - [GetController](#)
 - [SendCommand](#)
 - [ReadData](#)
 - [SendQuery](#)
-

<		>	
	2015 Teradyne, Inc. All rights reserved.		

gpib_apps.4.04

<	>
-----------------	-----------------

[GPIB Examples](#) : [GPIB Functions](#) : Define Constants for Carriage Return and Line Feed

Define Constants for Carriage Return and Line Feed

[Public Const](#) CR As Integer = &HD

[Public Const](#) LF As Integer = &HA

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

gpib_apps.4.05

<	>
-----------------	-----------------

[GPIB Examples](#) : [GPIB Functions](#) : Create Variables to Hold the Read and Write Terminators

Create Variables to Hold the Read and Write Terminators
Some equipment uses different terminators for read and write operations.
[Private](#) RTerminator As String
[Private](#) WTerminator As String

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

gpib_apps.4.06



[GPIB Examples](#) : [GPIB Functions](#) : ConnectDevice

ConnectDevice

This function accepts parameters for a device on a GPIB board and establishes a connection to this device. You must specify the board number, and the address of the device. Additionally you must specify:

- The read and write terminators as Strings
- A Boolean variable that indicates whether EOI is used to terminate operations
- A Double value, that indicates the timeout, in seconds, for GPIB operations

The connect operation creates a device descriptor in the controller that holds all information necessary to communicate with a GPIB-based device. This information is used by read/write operations.

The operation can fail if an address (on the board specified by boardNum) is already used. This can occur if another device has been connected using the same boardNum or address value, either from an earlier call to this function, or by an entry in the Tester Configuration file.

If the call succeeds, the address value is no longer available for other connect calls on the board specified by boardNum.

Public Function ConnectDevice(ByVal boardNum As Integer, ByVal addr As Long,
ByVal RTerm As String, ByVal WTerm As String,
ByVal EOVal As BOOL, ByVal DevName As String,
ByVal TimeoutVal As Double) As Boolean

Dim ctrl As DriverGPIBController

ConnectDevice = False

,

'First obtain a controller for the board number. If a controller exists,
'set the error trap and write the string to the device.

,

Set ctrl = GetController(boardNum)

If (Not (ctrl Is Nothing)) Then

On Error GoTo CtrlErr

Call ctrl.Connect(addr, RTerm, WTerm, EOIVal, DevName, TimeoutVal)

On Error GoTo 0

ConnectDevice = True

End If

Exit Function

CtrlErr:

' Add your own error processing here.

On Error GoTo 0

End Function

gpib_apps.4.07

	
---	---

[GPIB Examples](#) : [GPIB Functions](#) : DisconnectDevice

DisconnectDevice

This function frees device configuration for a particular device and makes the address, on the board specified by boardNum, available again. Subsequent GPIB operations on the boardNum or address pair will fail.

Public Function DisconnectDevice(ByVal boardNum As Integer, ByVal addr As Long) As Boolean

```
Dim ctrl As DriverGPIBController
```

```
DisconnectDevice = False
```

```
Set ctrl = GetController(boardNum)
```

```
If (Not (ctrl Is Nothing)) Then
```

```
    On Error GoTo CtrlErr
```

```
        Call ctrl.Disconnect(addr)
```

```
    On Error GoTo 0
```

```
    DisconnectDevice = True
```

```
End If
```

```
Exit Function
```

```
CtrlErr:
```

```
    ' Add your own error processing here.
```

```
    On Error GoTo 0
```

End Function

<		>
	2015 Teradyne, Inc. All rights reserved.	

gpib_apps.4.08

<	>
----------------------	----------------------

[GPIB Examples](#) : [GPIB Functions](#) : [GetController](#)

GetController

This function accepts a GPIB board number (0 to N) as an argument and returns a GPIB Controller language interface, if any exists. The interface can be used to communicate with a device connected to the board specified by the boardNum argument.

You must test the function return to verify that a valid language interface has been returned.

Public Function GetController(ByVal boardNum As Long) As DriverGPIBController

```
Set GetController = Nothing      ' Assume that no controllers exist.
```

```
,
```

```
' Access GPIB Service properties and methods through TheHdw.GPIB.
```

```
' The count property indicates how many different controllers have been
```

```
' created. There will be one controller per GPIB board or per board entry
```

```
' in the GPIB section of the tester configuration file.
```

```
,
```

```
If (TheHdw.GPIB.Count > 0) Then  ' If no controllers exist, just exit
```

```
    ' the function.
```

```
    On Error GoTo CtrlErr        ' Trap any errors
```

```
,
```

```
' The default property of the GPIB language interface is the item
```

```
' property. This statement could also have been written
```

```
' TheHdw.GPIB.item(boardNum).
```

```
,
```

Set GetController = TheHdw.GPIB(boardNum)
On Error GoTo 0

End If
Exit Function

CtrlErr:

' Add your own error processing here.

On Error GoTo 0

End Function

		
	2015 Teradyne, Inc. All rights reserved.	

gpib_apps.4.09

<	>
---	---

[GPIB Examples](#) : [GPIB Functions](#) : SendCommand

SendCommand

This function accepts a board number (that corresponds to a GPIB controller), the address of a device on a controller, and a text string to send to the device. The controller interface for the board is obtained and the string is written to the address.

Be sure to trap errors, as GPIB operations may result in errors (like no device present at the address).

Public Function SendCommand(ByVal GPIBBoard As Integer, ByVal addr As Long, ByVal cmd As String) As Boolean

```
Dim ctrl As DriverGPIBController
```

```
SendCommand = False
```

```
' First obtain a controller for the board number. If a controller exists
' set the error trap and write the string to the device. The controller
' automatically handles termination according to the specifications of the
' device in the Tester Configuration file or according to the
' specifications used in the ConnectDevice call.
'
```

```
Set ctrl = GetController(GPIBBoard)
```

```
If (Not (ctrl Is Nothing)) Then
```

```
    On Error GoTo CtrlErr
```

```
        Call ctrl.write(addr, cmd)
```

```
    On Error GoTo 0
```

SendCommand = True

End If

Exit Function

CtrlErr:

' Add your own error processing here.

On Error GoTo 0

End Function

<	>	
	2015 Teradyne, Inc. All rights reserved.	

gplib_apps.4.10



[GPIB Examples](#) : [GPIB Functions](#) : ReadData

ReadData

This function reads data from the GPIB.

Public Function ReadData(ByVal boardNum As Integer, ByVal addr As Long, ByRef ReplyBuf As String) As Boolean

```
Dim ctrl As DriverGPIBController
```

```
ReadData = False
```

```
Set ctrl = GetController(boardNum) ' Get the controller. If it doesn't exist, exit.
```

```
If (Not (ctrl Is Nothing)) Then Exit Function
```

```
On Error GoTo CtrlReadErr
```

```
Call ctrl.Read(addr, ReplyBuf) ' Attempt to read data from the GPIB
```

```
On Error GoTo 0
```

```
ReadData = True
```

```
Exit Function
```

CtrlReadErr:

```
' Implement your error processing here.
```

```
On Error GoTo 0
```

End Function

<		>
	2015 Teradyne, Inc. All rights reserved.	

gpib_apps.4.11

<	>
---	---

[GPIB Examples](#) : [GPIB Functions](#) : SendQuery

SendQuery

This function writes a command (that expects to return data) and then reads the reply.

This function assumes that the device doesn't assert an SRQ before the reply can be read, and it makes no allowance for processing time on the part of the device.

Public Function SendQuery(ByVal boardNum As Integer, ByVal addr As Long, ByVal cmd As String, ByRef ReplyBuf As String) As Boolean

```
Dim ctrl As DriverGPIBController
```

```
SendQuery = False
```

```
' Send the command. This function has not been implemented in terms  
' of SendCommand and ReadData because each of these functions calls  
' GetController. We don't need redundant calls to GetController.
```

```
Set ctrl = GetController(boardNum)
```

```
If (Not (ctrl Is Nothing)) Then Exit Function
```

```
On Error GoTo CtrlWriteErr
```

```
Call ctrl.write(addr, cmd) ' Write the command to the device.
```

```
On Error GoTo 0
```

```
On Error GoTo CtrlReadErr
```

Call ctrl.Read(addr, ReplyBuf) ' There was no error on the write, call read function.

On Error GoTo 0

SendQuery = True
Exit Function

CtrlWriteErr:

' Implement your error processing here.
On Error GoTo 0
Exit Function

CtrlReadErr:

' Implement your error processing here.
On Error GoTo 0
Exit Function

End Function

gpib_apps.4.12



GPIB Examples : Using GPIB Functions

Using GPIB Functions

The following subroutine illustrates calling these functions.

Sub CallGPIBFunctions()

```
'  
' First connect to the device at address 5, on board 0.  
'  
If (True = ConnectDevice(0, 5, Chr(CR), Chr(LF), True, "MyDevice", 0.3)) Then  
    MsgBox "Connect to address 5 succeeded"  
Else  
    MsgBox "Connect to address 5 failed"  
End If  
'  
' Now send the device an "id?" command and get the reply.  
'  
ReplyBuf = Space(50)  
If (True = SendQuery(0, 5, "id?", ReplyBuf)) Then  
    MsgBox "SendQuery to address 5 SUCCEEDED"  
Else  
    MsgBox "SendQuery to address 5 FAILED"  
End If  
If (True = DisconnectDevice(0, 5)) Then  
    MsgBox "Disconnecting address 5 Succeeded"  
Else  
    MsgBox "Disconnecting address 5 Failed"  
End If
```

```
'  
' Try to select an invalid controller.  
'  
  
Set ctrl = GetController(10)  
If (ctrl Is Nothing) Then  
    MsgBox "No controller for board number 10"  
End If  
'  
  
' Now send the device on board 0, address 5 an "id?" command and get the  
' reply. This command fails since the device has been disconnected.  
'  
  
ReplyBuf = Space(50)  
If (False = SendQuery(0, 5, "id?", ReplyBuf)) Then  
    MsgBox " SendQuery to disconnected device at address 5 Failed"  
End If  
End Sub
```

gpib_prog.3.1

<	>
----------------------	----------------------

GPIB Programming

GPIB Programming

Program GPIB instruments using the following VBT statement:

`Thehdw.GPIB(Optional boardNum = 0).xxx`

The GPIB board number (boardNum) is an optional parameter. Only one GPIB board can be in the test system and boardNum must be 0 so you do not need to specify boardNum. For example:

`TheHdw.GPIB.xxx.`

where xxx is the GPIB address of the instrument to be programmed.

The following methods are properties used with the GPIB controller board or the GPIB instrument specified by boardNum. The syntax of the methods and properties are described in both VB and C++ form, with the C++ form in brackets.

The following topics are available:

- [Send a GPIB Command or Data](#)
- [Receive Data From the GPIB Instrument](#)
- [Serial Poll on a GPIB Instrument](#)
- [Wait Until the GPIB Service Request Line is Asserted](#)
- [Translate a GPIB Instrument Name to Its GPIB Address](#)
- [System Reset](#)

- [Tester Configuration File](#)

For more information, see:

- [GPIB \(VBT Syntax Reference\)](#)

- [GPIB Examples](#)

<

>

gpib_prog.3.2

<	>
-----------------	-----------------

[GPIB Programming](#) : Send a GPIB Command or Data

[Send a GPIB Command or Data](#)

[SendData\(instr As Long, command As String\)](#)

[\[HRESULT SendData\(Long instr, BSTR command\)\]](#)

[This method sends command or data through the GPIB bus to the GPIB instrument specified by instr. If used offline, just return S_OK.](#)

instr	GPIB address (long) or alias (string) that you assigned to the GPIB instrument.
command	Command (string) to send to the GPIB instrument.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

gpib_prog.3.3

<	>
-----------------	-----------------

GPIB Programming : Receive Data From the GPIB Instrument

Receive Data From the GPIB Instrument

ReceiveData(instr As Long) As String
[HRESULT ReceiveData(Long instr, BSTR* data)]

This method receives data through the bus from the GPIB instrument specified by instr. If used offline, sets data to an empty string and returns S_OK;

instr	GPIB address (Long) or alias (String) that you assigned to the GPIB instrument.
data	Data (String) received from the GPIB instrument.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

gpib_prog.3.4

<	>
-----------------	-----------------

GPIB Programming : Serial Poll on a GPIB Instrument

Serial Poll on a GPIB Instrument

SerialPoll(Variant instr) As Byte

[HRESULT SerialPoll(Variant instr, BYTE* status)]

This method performs a serial poll on a GPIB instrument specified by instr and returns the status of the GPIB instrument.

instr	GPIB address (Long) or alias (String) that you assigned to the GPIB instrument.
status	The status byte of the GPIB instrument.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

gpib_prog.3.5

<	>
--------------------------------------	--------------------------------------

[GPIB Programming](#) : Wait Until the GPIB Service Request Line is Asserted

[Wait Until the GPIB Service Request Line is Asserted](#)

[WaitSRQ\(\)](#) As Boolean

[\[HRESULT WaitSRQ\(VARIANT_BOOL* srq\)\]](#)

[This method waits until the GPIB Service Request \(SRQ\) line is asserted or the timeout period has expired. It returns true if the GPIB SRQ line is asserted or false otherwise.](#)

<	>
--------------------------------------	--------------------------------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

gpib_prog.3.6

	
---	---

[GPIB Programming](#) : Translate a GPIB Instrument Name to Its GPIB Address

[Translate a GPIB Instrument Name to Its GPIB Address](#)

[NameToAddress\(String name\)](#) As Long

[\[NameToAddress\(BSTR name, long* addr\)\]](#)

This method translates a GPIB instrument name into its GPIB address based on the GPIB information in the configuration file.

Properties are listed below.

[Set or Get the Error Mode](#)

[ErrMode](#) As tIGPIBErrMode

[\[HRESULT ErrMode\(tIGPIBErrMode ErrorMode\)\]](#)

[\[HRESULT ErrMode\(tIGPIBErrMode* ErrorMode\)\]](#)

[Gets or sets the error mode for the GPIB bus communication.](#) tIGPIBErrMode is an enumeration with tIGPIBErrModeOn and tIGPIBErrModeOff as valid values. When ErrMode is tIGPIBErrModeOff, GPIB bus communication is not checked for errors. When ErrMode is tIGPIBErrModeOn, GPIB bus communication is checked for errors and errors are returned by the Error Service. The default ErrMode is tIGPIBErrModeOn.

[Set or Get the Timeout](#)

[Timeout](#) As Double

[\[HRESULT Timeout\(double timeout\)\]](#)

[\[HRESULT Timeout\(double* timeout\)\]](#)

[Gets or sets the general timeout value \(in seconds\) for the GPIB bus communication.](#) The default timeout value is 10s. If a GPIB instrument does not specify its own timeout value for the GPIB bus communication, the timeout value specified by this property is used. If a GPIB instrument specifies its own timeout value, then the instrument's timeout value is used during its GPIB bus communication. After the instrument finishes, the timeout value for the GPIB bus communication is set back to the value of this property.

Get the GPIB Addresses

InstrAddresses() As Long()

[HRESULT InstrAddresses(SAFERAY(long)* InstrAddrs)]

Gets GPIB addresses for all the mapped GPIB instruments on the current GPIB board. This can be a hidden method that returns an array of primary GPIB addresses.

When the production operating interface instantiates GPIB based handler or prober drivers, you can use this property to check for address conflicts between GPIB instrument and the handler or the prober drivers. The handler or prober or both drivers use the GPIB controller to communicate with the GPIB bus. The GPIB controller allows the GPIBAddr property to be modified to check if the entered value is one of the mapped addresses obtained by this method. If it is not, then the GPIB controller sets the GPIBAddr for the handler or prober. If it is one of the mapped addresses, then the GPIB controller returns an address conflict error and tells you to resolve the conflict.

	
---	---

gpib_prog.3.7

<	>
-----------------	-----------------

GPIB Programming : System Reset

System Reset

Each GPIB instrument or equipment should reset itself individually. General test system commands that reset the bus can adversely affect handlers or probers or both.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

gpib_prog.3.8

GPiB Programming : Tester Configuration File

Tester Configuration File

Once an instrument's GPIB address is set, the instrument is ready to respond to commands from the GPIB driver. Before the GPIB driver can send commands to instruments, however, it must know what instruments are on the bus, their GPIB addresses, and their capabilities. It derives this information from a tester configuration file.

A GPIB section in a tester configuration file corresponds to a physical GPIB board. It identifies the instruments that reside on a board. Each board has a separate GPIB section. Each physical device has a line in the GPIB section.

The tester configuration file is in:

C:\Program Files\Teradyne\IG-XL\N.nn.nn\bin\TesterConfig.txt

For information on creating and editing a tester configuration file, see [Tester Configuration](#).

Example Tester Configuration File Entries for GPIB Instruments

The following example shows tester configuration file entries for GPIB that include three GPIB boards, two National Instruments boards and one AT&T board.

```
<GPiB CNIGPiBController0 0>
```

```
# Addr LName PName  RT WT  EOI Timeout DIBSlot CableFile
22 GPIBInst  PhysInst CR CRLF F  0.5   1    PhysInstCable.txt
23 PMO      HP3458A  LF CR  F  0.75  1    PMOCableAddr23.txt
```

```
</GPiB CNIGPiBController0 0>
```

```
<GPiB ATT0 1>
```

```
# Addr LName PName  RT WT  EOI Timeout DIBSlot CableFile
15 ABC     ABCPI   LF CR  T  0.75  1    OPMCableAddr15.txt
6  Unsplnst MyInst  CR CRLF F
```

```
</ GPiB ATT0 1>
```



```
<GPIB CNIGPIBController1 2>
# Addr LName PName  RT  WT   EOI Timeout DIBSlot CableFile
22 GPIBInst  PhysInst CR  CRLF F   0.5   1     PhysInstCable.txt
25 DEF      DEFPI   LF CR  F   0.75  1     MOPCableAddr25.txt
</ GPIB CNIGPIBController1 2>
```

This example includes the following instruments:

PMO	Precision Meter Option
ABC	Example Teradyne instrument
DEF	Example Teradyne instrument
GPIBInst	Example GPIB entry
Unsplnst	Unsupported instrument

The [vendor names](#) CNIGPIBController and ATT must match the names of the controllers in the GPIB service’s key, GPIB_CONTROLLERS. The number following the vendor tag is the vendor specific board ID. The vendor board ID is used to configure the controller after it has been created. The GPIB [<#>](#) tag specifies the Teradyne board identifier. This is the order in which boards appear in the GPIBService language interface.

The configuration information is read and sorted by logical instrument. All “Attached” logical instruments [receive configurations for all physical instruments that they control](#). [Unsupported instruments, such as](#) Unsplnst/MyInst, reside in the configuration file. A connection to an unsupported instrument is created in the controller and is passed to the controller. However, no other Teradyne services or drivers receive this information.

[Other tester configuration file entries are:](#)

CNIGPIBController0	For Teradyne use only. GPIB 0 is the address of the GPIB controller.
Addr	Is for PrimaryAddress. Address and Primary address are used interchangeably here. GPIB address 0 is reserved for GPIB controller board. GPIB instruments can use an address between 1 and 30. The entry in this column must be unique because no two GPIB instruments can be connected to the same primary and secondary addresses at the same time. IG-XL checks for GPIB address conflicts during GPIB system mapping and reports an error if GPIB addresses are not unique.

LName	Is the logical GPIB instrument name for a Teradyne supported GPIB instrument. For a GPIB instrument not supported by Teradyne, LName is an alias that can be used instead of the GPIB address to access the instrument through VBT language.
PName	Is the physical GPIB instrument name for a Teradyne supported GPIB instrument and the actual instrument name for a non-Teradyne supported GPIB instrument.
RT	Is the Read Terminator for GPIB data transfers of the GPIB instrument. It signals the end of a read (receive data) when the read terminator character is transmitted
WT	Is the Write Terminator for GPIB data transfers of the GPIB instrument. It signals the end of a write (send data) when the write terminator character is transmitted.
EOI	Signals whether the GPIB EOI line is asserted with the last byte of a data transfer for GPIB data transfers. T (True) means the GPIB EOI line is asserted at the end of GPIB data transfers, and F (False) means the GPIB EOI line is not asserted at the end of GPIB data transfers. GPIB data transfers are terminated either when the GPIB EOI line is asserted with the last byte of a transfer or when the read terminator/write terminator character is transmitted.
Timeout	Is optional. For a Teradyne supported GPIB instrument that has a value in this column, it specifies the timeout value in seconds for GPIB operation.
DIBSlot	Is optional. For a Teradyne supported GPIB instrument that has a value in this column, it specifies the slot on the DIB that the instrument is connected to.
CableFile	Is optional. For a Teradyne supported GPIB instrument that has a value in this column, it specifies the channel and signal information of the instrument.

<

>

gpib_too.2.1

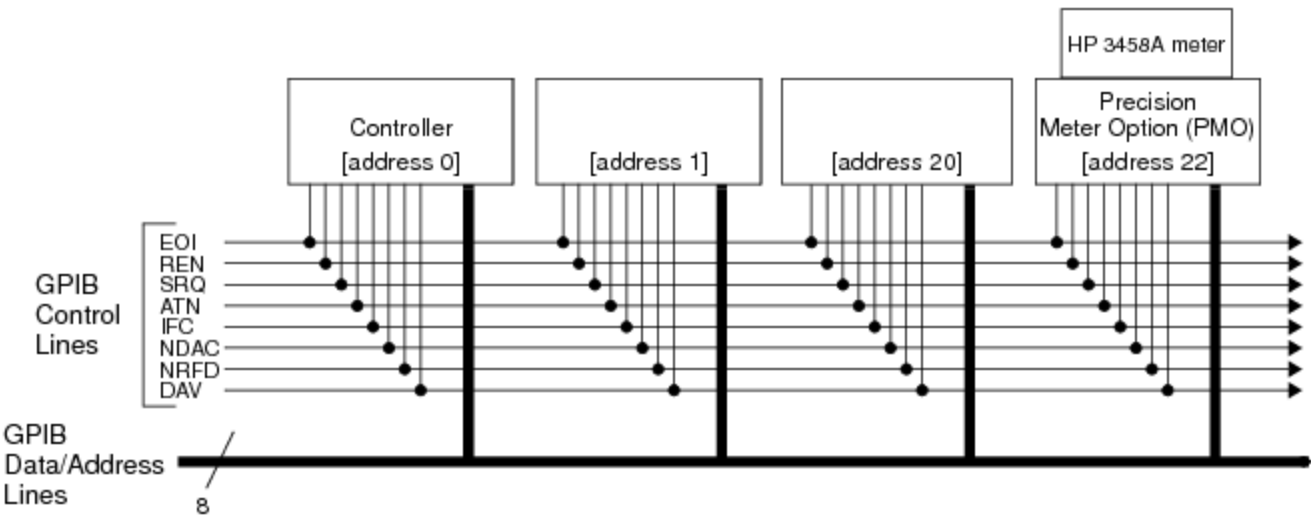
<

>

GPIB Theory of Operation

GPIB Theory of Operation

The GPIB consists of eight control lines and eight data lines. Instruments connected to the GPIB share the bus.



General Purpose Interface Bus Lines

The GPIB data/address lines carry three types of information:

- GPIB commands
- GPIB addresses
- Instrument commands and data

Each instrument on the bus can talk or listen for messages or ignore the bus, with the GPIB driver regulating and managing the communications between instruments. Only one instrument at a time can talk. The other instruments must either listen to the talker or ignore the talker. In most cases,

the GPIB controller is talking or listening to one or more instruments. This is because the controller acts as the intermediary between a test program and the GPIB.

< >		
	2015 Teradyne, Inc. All rights reserved.	

gpib_too.2.2

<

>

GPIB Theory of Operation : GPIB Terminology

GPIB Terminology

GPIB	An acronym for the General Purpose Interface Bus. It is an eight-bit, byte serial communications interface with a set of associated control lines.
IEEE 488.2	A communications specification for communication on the GPIB.
Terminator	A character or bus line that is used to indicate end of transmission. The terminator is usually a carriage return (CR), line feed (LF), or carriage return with line feed (CRLF) character.
EOI	A GPIB bus control line that can be asserted to indicate end of transmission. This line can be used with or without termination characters.
EOS Mode	The termination mode in which the GPIB software operates. Depending on the mode the EOI line can be asserted when the termination character is detected or a read can be terminated when the last character of the terminator is read. This mode also determines whether seven or eight-bit comparison is used to detect the terminator.
SRQ	The GPIB line that a device asserts to indicate that it requires service. SRQ is also known as Service Request.
RQS	A request for service. The act of requesting service requires the assertion of the SRQ line. The SRQ line can sometimes be referred to the RQS.
Status Byte	This is a data byte defined by the IEEE 488.2 standard. When a device requests service it sets bit 6 in the status byte. It may also encode other

	information in other bits of the status byte. The value of the status byte is retrieved when the controller performs a serial poll.
Serial Poll	The process of polling a device and reading the status byte. Serial polling is performed one instrument at a time.
Listener	A GPIB device that receives data messages from a talker.
Talker	A GPIB device that sends data messages to a listener.
Timeout	A user specified period of time that prevents I/O functions from hanging indefinitely if a problem occurs on the GPIB.
UNL	The un-listen bus command. This used to un-address any active listeners.
UNT	The un-talk bus command. This command is used to un-address any active talkers.

< >	
	2015 Teradyne, Inc. All rights reserved.

gpib_too.2.3

<

>

GPIB Theory of Operation : GPIB Commands

GPIB Commands

The GPIB has a command set, listed in the table below. The GPIB software gives you access to two of these commands:

Go to Local (GTL)	Causes the currently addressed listeners to return to local programming mode.
Selected Device Clear (SDC)	Resets all the listeners. The reset state for each listener is instrument dependent. (Refer to its user’s manual.)

GPIB software manages the other commands. The table is provided for reference only.

GPIB Commands

Command Type	Octal Value	GPIB Command
Addressed Commands	001	Go To Local (GTL)
	004	Selected Device Clear (SDC)
	005	Parallel Poll Configure (PPC)
	010	Group Execute Trigger (GET)
	011	Take Control (TCT)
Universal Commands	021	Local Lockout (LLO)
	024	Device Clear (DCL)
	025	Parallel Poll Unconfigure (PPU)

030	Serial Poll Enable (SPE)
031	Serial Poll Disable (SPD)
077	Unlisten (UNL)
137	Untalk (UNT)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

gpib_too.2.4

<

>

GPIB Theory of Operation : GPIB Addresses

GPIB Addresses

The GPIB driver identifies each instrument on the bus (including the controller) using its preset GPIB address. Instruments usually acquire a GPIB address at the factory, which can be reset using an address switch, jumpers, or a control panel. (Refer to the instrument’s user manual for an explanation of how to set its GPIB address.)

An instrument’s address is actually two addresses: a talk address and a listen address. If an instrument detects its listen address on the GPIB, it begins listening for information on the bus. If an instrument detects a talk address on the GPIB and has something to say, it begins talking. The high-level GPIB language eliminates talk and listen addresses and instead refers to instruments using a single decimal address. The table lists the decimal GPIB addresses along with the corresponding, low-level talk and listen address codes. The table also identifies addresses allocated to Teradyne instruments.

GPIB Addresses

High-Level Addresses (decimal)	Low-Level Addresses: Listen/Talk (octal)	Instrument at That Address
address 0	040/100	GPIB Controller
address 1	041/101	
address 2	042/102	
address 3	043/103	
address 4	044/104	
address 5	045/105	

address 6	046/106
address 7	047/107
address 8	050/110
address 9	051/111
address 10	052/112
address 11	053/113
address 12	054/114
address 13	055/115
address 14	056/116
address 15	057/117
address 16	060/120
address 17	061/121
address 18	062/122
address 19	063/123
address 20	064/124
address 21	065/125
address 22	066/126
address 23	067/127
address 24	070/130
address 25	071/131
address 26	072/132
address 27	073/133
address 28	074/134
address 29	075/135
address 30	076/136

Precision Meter Option (PMO)

The GPIB has 31 available addresses, but it can support a maximum of only 19 instruments. Four addresses are assigned to Teradyne instruments, leaving 15 GPIB addresses for other instruments.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

gpib_too.2.5

<

>

GPIB Theory of Operation : GPIB Control Lines

GPIB Control Lines

GPIB software uses the GPIB control lines to manage an orderly flow of information across the bus. Although this control activity is transparent to the test system programmer, knowing the function of these control lines provides a better understanding of GPIB programming statements and their parameters.

The GPIB control lines are as follows:

ATN	Attention. When this signal is true, signals on the GPIB data/address lines are interpreted as either GPIB addresses or GPIB commands. When this signal is false, signals on the GPIB data/address lines are interpreted as data intended for listeners.
DAV	Data Valid. When true, this signal indicates that the information on the data/address lines is valid.
EOI	End or Identify. When ATN is false and this signal is true, it indicates the end of a transmission. That is, the current byte on the data/address lines represents the last byte of the commands or data.
IFC	Interface Clear. When true, this signal halts bus communications. Communications resume when IFC is false.
NDAC	Data Not Accepted. Listeners use this line to indicate when they have accepted (false) or have not accepted (true) the information being transferred.
NRFD	Not Ready For Data. Listeners use this line to indicate when they are ready (false) or are not ready (true) to accept information.

REN	Remote Enable. When true, this signal places all listeners capable of remote operation in remote programming mode. When false, it returns all instruments to local programming mode.
SRQ	Service Request. Instruments use this line to request the controller's attention. Typically, this occurs when an instrument has data ready to transmit or when it detects an error condition.

The control lines are active low, that is, the signal is true when its voltage is low (near 0V), and the signal is false when its voltage is high (near 5V).

<

>

2015 Teradyne, Inc. All rights reserved.